Predicting Loan Default Losses:

A Machine Learning Approach to Credit Risk Assessment

Gabriel Augusto Arsego

Manpreet Kaur

Kight Pruszynski

Master of Science in Business Analytics, Kent State University

Spring 2026 ADVANCED DATA MINING AND PREDICTIVE ANALYTICS
(BA-64037-001)

May 3, 2026

Group participant contributions

Gabriel Augusto Arsego	Project management Data cleaning and preprocessing Project presentation
Kight Pruszynski	Linear modeling and feature analysis Group report and R cohesivity revision
Manpreet Kaur	Iteration Nonlinear prediction Project report

Abstract

This study develops and evaluates machine learning models for predicting financial losses from loan defaults. Unlike traditional binary classification approaches that categorize loans as simply 'good' or 'bad', our methodology predicts continuous loss amounts, providing more granular risk assessment for financial institutions. Using a dataset of 80,000 loan records with 763 features, we implemented a comprehensive analytical pipeline encompassing data preprocessing, feature engineering, and multi-model evaluation.

Our preprocessing methodology addressed 587,747 missing values through median imputation, removed 11 zero-variance features, and applied standardized scaling with outlier capping. Feature engineering included correlation analysis, polynomial term generation, and interaction feature creation. We evaluated seven distinct modeling approaches: baseline linear regression, Lasso regression, Ridge regression, Elastic Net, Random Forest, XGBoost, and an enhanced linear model incorporating interaction effects.

XGBoost achieved the best performance with a validation Mean Absolute Error (MAE) of 1.425, marginally outperforming Lasso (1.428) and Elastic Net (1.428). However, all models exhibited low R-squared values (0.3-0.7%), indicating that loan losses contain substantial inherent variability not captured by available features. Lasso's automatic feature selection reduced dimensionality from 731 to 88 features, identifying key predictors including f471, f536, and f674. An ensemble combining Lasso, XGBoost, and Random Forest was constructed using inverse-MAE weighting, producing predictions for 25,471 test cases.

While predictive accuracy is constrained by data limitations, the models demonstrate practical value for risk assessment, portfolio management, and loan pricing decisions. The analysis reveals that financial loss prediction requires integration of macroeconomic factors and temporal dynamics not present in static borrower characteristics alone.

1. Introduction

This project asks us to determine whether a loan will default, as well as the loss incurred if it does default.

1.1 Background and Motivation

Credit risk assessment represents one of the most consequential applications of predictive analytics in the financial services industry. Financial institutions extend trillions of dollars in credit annually, making accurate loss prediction essential for pricing loans appropriately, maintaining adequate capital reserves, and ensuring portfolio stability. Traditional credit risk models have relied primarily on binary classification—distinguishing between loans likely to default and those expected to perform normally. While useful for approve/deny decisions, this binary framework provides insufficient granularity for modern risk management practices that require precise loss amount estimation.

The 2008 financial crisis dramatically illustrated the limitations of simplistic risk models. Many institutions underestimated not just the probability of default but, more critically, the severity of losses when defaults occurred. This experience catalyzed a shift toward more sophisticated approaches that predict continuous loss amounts rather than binary outcomes. Such models

bridge traditional banking perspectives (focused on capital adequacy) with asset management viewpoints (concerned with return optimization given risk constraints).

Machine learning techniques offer substantial advantages for this task. Unlike conventional statistical models that impose rigid parametric assumptions, machine learning algorithms can discover complex non-linear patterns and interaction effects in high-dimensional feature spaces. Ensemble methods can combine multiple model perspectives, potentially improving both accuracy and robustness. Feature selection algorithms can identify the most predictive variables from hundreds of potential candidates, improving interpretability while reducing overfitting.

1.2 Problem Statement

This research addresses the challenge of predicting monetary losses from loan defaults using machine learning techniques. Specifically, given a set of loan and borrower characteristics measured at origination, we aim to predict the loss amount that will occur if the loan defaults. This differs from conventional credit scoring, which typically predicts default probability, and from loss-given-default (LGD) models, which estimate loss conditional on default having occurred. Our approach integrates both components into a single predictive framework.

The dataset comprises 80,000 historical loan records with 763 features representing borrower demographics, loan terms, and financial metrics. The target variable represents realized losses ranging from near-zero to substantial amounts, exhibiting right-skewed distribution typical of financial loss data. The prediction task involves building models on training data and generating loss predictions for 25,471 test cases where outcomes are unknown.

1.3 Objectives

The specific objectives of this research are:

Develop a robust data preprocessing pipeline that handles missing values, scales features appropriately, and identifies irrelevant variables

Engineer meaningful features including polynomial terms and interaction effects to capture non-linear relationships

Implement and compare multiple machine learning algorithms spanning linear models (OLS, Lasso, Ridge, Elastic Net), tree-based methods (Random Forest, XGBoost), and enhanced approaches incorporating feature interactions

Evaluate models using appropriate metrics (Mean Absolute Error, RMSE, R-squared) on held-out validation data

Construct an ensemble model combining strengths of individual approaches

Generate predictions for test data and analyze model performance, limitations, and business applications

2. Literature Review and Theoretical Framework

Credit risk modeling has evolved substantially over recent decades. Early approaches relied on expert judgment and simple heuristics, eventually progressing to logistic regression models like Altman's Z-score (Altman, 1968). These discriminant analysis techniques provided systematic frameworks but imposed strong linearity assumptions and struggled with complex feature interactions.

The Basel II Accord formalized the distinction between probability of default (PD), loss given default (LGD), and exposure at default (EAD) as key risk components. While this framework improved regulatory capital calculations, it treated these components independently despite their potential interdependencies. Modern approaches increasingly recognize that default probability and loss severity are often correlated—the same economic conditions that increase default likelihood may also reduce recovery rates.

Machine learning methods have gained prominence in credit risk modeling over the past two decades. Baesens et al. (2003) demonstrated that neural networks and support vector machines could outperform traditional logistic regression for default prediction. More recently, gradient boosting methods like XGBoost have shown exceptional performance across various financial prediction tasks (Chen & Guestrin, 2016). These ensemble methods iteratively build decision trees, each correcting errors from previous iterations, effectively capturing non-linear patterns and variable interactions.

Regularization techniques, particularly Lasso (L1) and Ridge (L2) regression, address overfitting in high-dimensional settings where features outnumber observations (Hastie et al., 2009). Lasso's ability to perform automatic feature selection by shrinking irrelevant coefficients to exactly zero makes it particularly valuable for interpretability. Elastic Net combines both penalties, balancing Lasso's sparsity with Ridge's stability (Zou & Hastie, 2005).

Despite these advances, financial loss prediction remains challenging. Thomas et al. (2017) note that credit losses exhibit substantial random variation beyond what borrower characteristics can explain. Macroeconomic factors, industry shocks, and idiosyncratic events all contribute variance that static models struggle to capture. This inherent uncertainty limits achievable prediction accuracy regardless of algorithmic sophistication.

3. Data Description and Exploratory Analysis

3.1 Dataset Overview

The dataset consists of two components: a training set with 80,000 observations and a test set with 25,471 observations. Each observation represents a single loan with 763 predictor variables and one target variable. The predictor variables are labeled f1 through f777 (with some gaps in numbering), representing various loan and borrower characteristics. The anonymization of feature names prevents direct interpretation but is common in competitive datasets to protect proprietary information.

Initial inspection revealed diverse data types including integers (394 variables), numeric floating-point values (285 variables), 64-bit integers (47 variables), and character strings (37 variables). This heterogeneity required careful type handling during preprocessing. The training

set includes the target variable 'loss' representing monetary losses, while the test set omits this variable—our prediction target.

3.2 Target Variable Distribution

The loss variable exhibits characteristics typical of financial loss distributions: substantial right skewness with a long tail of extreme values. Summary statistics reveal:

• Mean: 0.831 indicating average loss across all loans
• Median: 0.772 below the mean, confirming right skewness
• Standard deviation: 4.15 demonstrating high variability
• Range: 0.08 to 8.45 with most values concentrated below 2.0
• Skewness: Positive, with tail extending toward high values

• Mean: 0.831 indicating average loss across all loans
• Median: 0.772 below the mean, confirming right skewness
• Standard deviation: 4.15 demonstrating high variability
• Range: 0.08 to 8.45 with most values concentrated below 2.0
• Skewness: Positive, with tail extending toward high values

This distribution pattern is economically sensible. Most defaulted loans result in moderate losses after recovery efforts (collateral liquidation, partial repayment), while a minority involve near-total loss. The high standard deviation relative to the mean (coefficient of variation = 5.0) signals substantial inherent variability, foreshadowing challenges in achieving high predictive accuracy.

3.3 Feature Characteristics

Exploratory analysis of predictor variables revealed several important patterns. Correlation analysis between each feature and the target variable identified weak linear relationships throughout. The strongest correlations were:

• f471: $r = 0.0502$ (strongest predictor)
• f536: $r = 0.0323$
• f674: $r = 0.0194$
• f67: $r = 0.0173$
• f617: $r = 0.0161$

Even the best predictor explains only 0.25% of variance individually ($r^2 = 0.0025$), suggesting that: (1) relationships may be non-linear, (2) predictive power emerges from feature combinations rather than individual variables, or (3) important information is missing from the dataset. This pattern is consistent with financial data where outcomes depend on complex interactions among many factors rather than single dominant predictors.

3.4 Missing Data Analysis

The dataset contained 587,747 missing values distributed across 292 features (38% of all features). Missing data patterns varied substantially:

Features with highest missingness rates included f662 and f663 (17.8% missing), f159 and f160 (17.7%), and f169 and f170 (17.5%). However, no feature exceeded 50% missing values, suggesting systematic data collection processes rather than random omissions. The target

variable had 50 missing observations (0.06% of training data), which were excluded from analysis since model training requires known outcomes.

Missing data mechanisms likely include: (1) Not applicable—certain features may not apply to all loan types, (2) Data not collected—some information may have been unavailable at origination, (3) Privacy redaction—sensitive fields may have been intentionally removed. Understanding these mechanisms would inform imputation strategy, but given anonymized features, we adopted a conservative median imputation approach that makes minimal distributional assumptions.

4. Methodology

4.1 Data Cleaning and Preprocessing Pipeline

Data preprocessing constitutes a critical phase ensuring model inputs are clean, consistent, and appropriately scaled. Gabriel took the reins on cleaning and preprocessing, leaning on methods he recently learned in Advanced Machine Learning. This step of the project posed incredible challenges right from the beginning, and the deeper Gabriel dove into it, the more challenging it became. The large size of the training and test data set, totalling over 100,000 rows, was a challenge in itself. Going into this, Gabriel knew he wouldn't have enough computational power to deal with such large data. After some discussions amongst group members and other students from class, the first solution Gabriel thought of was to prepare the data on Google Colab using Python after failed attempts in R Studio. Gabriel thought it was going to run smoothly since he wouldn't be dependent on his RAM, but the first coding created led to the crash of Colab and made it impossible for him to create the files that would be used for modeling. Colab reached its maximum capacity and only purchasing more data would allow him to go on with that first version.

The solution then was to keep only numeric variables, divide the train_v3.csv data set into chunks of 20,000, use downcast to reduce memory use, and then combine all the chunks. The id column was saved in a different file and kept out of the preprocessing. After that, in order to download the file without crashing Colab once again, Gabriel used correlation with loss as the target variable to reduce the number of variables to the 50 "best" ones in terms of prediction capabilities.

In [1]:

```python
import pandas as pd

chunk_size = 20000
chunks = []

for chunk in pd.read_csv("train_v3.csv", chunksize=chunk_size, low_memory=False):

    # Drop ID
    if "id" in chunk.columns:
        chunk = chunk.drop(columns=["id"])

    # Keep only numeric columns
    chunk = chunk.select_dtypes(include=['int64', 'float64'])

    # Fill missing values
    chunk = chunk.fillna(chunk.median())

    # Downcast to reduce memory
    for col in chunk.columns:
        if chunk[col].dtype == 'int64':
            chunk[col] = pd.to_numeric(chunk[col], downcast='integer')
        elif chunk[col].dtype == 'float64':
            chunk[col] = pd.to_numeric(chunk[col], downcast='float')

    chunks.append(chunk)

# Combine all cleaned chunks
df = pd.concat(chunks, ignore_index=True)

print(df.shape)
```

```
(80000, 742)
```

At this scale, computational power would not pose such a problem for any company that would have the minimum computational resources necessary for data analysis. But, we can see how on a much larger scale an approach like this could permit companies to deal with extremely large data sets without having too much problem.

Even with the files ready to use for modeling, Gabriel wanted to try a different approach. He was able to free space from his personal computer, which improved its performance when working with the dataset directly on R Studio. Therefore the cleaning and preprocessing were processed again, without using correlation for the variable selection, creating an entirely new cleaned file with what we believe to be a better and cleaner coding workflow.

After overcoming these computational constraints, our pipeline comprised five sequential stages:

Stage 1: Variable Type Filtering and Selection

Non-numeric variables were identified and removed from the modeling dataset. Regression algorithms require numeric inputs, and while categorical variables can be encoded (one-hot, ordinal, target encoding), the anonymization prevented understanding which variables were truly categorical versus corrupted numeric fields. To avoid introducing encoding errors, we conservatively retained only variables with inherently numeric types (integer, numeric, integer64).

This filtration reduced the feature space from 763 to 743 variables. The 20 removed variables included character strings, identifier fields, and date representations. While potentially informative if properly encoded, the risk of misspecification outweighed potential benefits given the anonymized context.

Stage 2: Missing Value Imputation

Missing values were imputed using median values calculated exclusively from the training set. Median imputation offers several advantages over alternatives:

First, median is robust to outliers and extreme values, which are common in financial data. Mean imputation would be biased by tail observations, while median represents the central tendency without distortion. Second, computing imputation statistics only from training data prevents information leakage from test data. Using test set statistics would invalidate the evaluation framework by incorporating knowledge of test outcomes into preprocessing. Third, median imputation is computationally efficient and stable, unlike more complex methods (k-nearest neighbors, multivariate imputation) that can be sensitive to scaling and outliers.

Alternative approaches considered but rejected included: (1) Mean imputation—vulnerable to outliers, (2) Mode imputation—appropriate for categorical but not continuous variables, (3) Forward/backward fill—requires temporal ordering not present in cross-sectional data, (4) Multiple imputation—computationally intensive with marginal expected improvement given weak feature correlations. We considered it might be worthwhile to apply k-nearest neighbors (KNN) for more precise correlated imputation had we more time.

After many attempts and errors, a clean result was achieved. The data set had no more missing values, which were filled with the median of each column and later used to fill the test set as well.

Final missing value checks:

```r
sum(is.na(df_final))
```

```
## [1] 0
```

```r
sum(is.na(test_final))
```

```
## [1] 0
```

```r
sum(is.nan(as.matrix(df_final)))
```

```
## [1] 0
```

```r
sum(is.nan(as.matrix(test_final)))
```

```
## [1] 0
```

Stage 3: Zero-Variance Feature Removal

Features with zero or near-zero variance provide no predictive information since they assume identical values across all observations. Eleven features met this criterion: f33, f34, f35, f37, f38, f678, f700, f701, f702, f736, and f764. These were removed for two reasons:

Mathematically, zero-variance features cause numerical issues during standardization (division by zero when computing z-scores). While this can be handled through conditional logic, retaining such features serves no purpose. Statistically, features with no variance contribute

nothing to model fitting—they cannot explain any variation in the target variable. Removing them simplifies the model without any loss of information.

Near-zero variance (standard deviation < 0.0001) was also flagged as potentially problematic. While these features have minimal variation, they don't cause numerical errors and might capture rare but important events. We retained near-zero variance features but applied capping during standardization to prevent extreme scaled values. Compared side-by-side, MAE results with and without near-zero variance features were the same, confirming the robustness of the models.

Stage 4: Feature Standardization and Outlier Management

Feature standardization (z-score normalization) transformed each variable to mean zero and unit variance using training set parameters:

$z = (x - \mu_train) / \sigma_train$

$z = (x - \mu_train) / \sigma_train$

where μ_train and σ_train represent the mean and standard deviation computed from training data. This transformation ensures:

- Scale invariance: Features measured in different units (dollars, percentages, counts) become comparable
- Algorithmic stability: Gradient descent converges faster when features are similarly scaled
- Regularization fairness: L1/L2 penalties affect all features equally rather than favoring smaller-scale variables
- Distance metric validity: Euclidean distances used in some algorithms become meaningful

Critical implementation details included:

First, we addressed division-by-zero errors. Features with standard deviation below 10^{-10} had their σ set to 1.0 before scaling. This prevents infinite or undefined scaled values while preserving the signal that variance is essentially zero (the feature remains at its mean value after transformation).

Second, we implemented outlier capping. Scaled values exceeding ±10 standard deviations were capped at ±10. This threshold is conservative—assuming normality, less than 1 in 10^{23} observations should exceed this range legitimately. Values beyond ±10 SD almost certainly represent data errors, extreme outliers, or numerical precision issues. Capping prevents these from dominating model fitting while retaining the information that the value is extreme.

Third, we verified numerical stability post-transformation. Any remaining infinite, NaN (not-a-number), or undefined values were replaced with zero. This serves as a final safety check ensuring the preprocessed data contains only finite numeric values.

Stage 5: Final Validation and Consistency Checks

Before proceeding to modeling, we performed comprehensive validation:

- Verified zero missing values remained after imputation
- Confirmed no infinite or NaN values present
- Checked that all features were numeric type

• Validated training and test sets had identical feature sets
• Ensured test set used training statistics for all transformations
• Confirmed target variable present only in training set

The final preprocessed datasets comprised 79,950 training observations and 25,471 test observations, each with 731 features (down from 763 original variables). All subsequent modeling used these standardized inputs.

Inspecting the naming and order of variables in the data set:

One thing Gabriel noticed when looking at the data set and talking to other groups is that there are “missing columns” or the columns are not numbered in the right order. They might have been removed or they were simply named like that.

As you can see in the image below, taken from the original training data set, the columns are not following the expected numerical order (e.g. going from f1 to f3, and from f9 to f13). That didn’t impact the cleaning and preprocessing.

	V1	id	f1	f3	f4	f5	f6	f7	f8	f9	f13	f14	f15	f16	f17	f18	f19	f20
1	78539	78539	120	0.146251127	2200	4	76878	8703	724	118.93	14	0.7417	0.7177	11447	0.7236	0.7177	0.4344	0.7177
2	61541	61541	154	0.349399073	4200	4	76635	2843	3253	158.08	4	0.7241	0.7241	2211576	0.7241	0.7119	0.5068	0.7241
3	76531	76531	126	0.975969168	1500	10	7399	437	1453	123.31	15	0.6552	0.6441	1755652	0.6441	0.6441	0.3679	0.6441
4	22066	22066	127	0.951302949	3100	7	14448	2681	2136	128.62	11	0.8034	0.7460	2107744	0.7705	0.7231	0.5738	0.7231
5	45589	45589	162	0.518485643	3500	7	80502	3840	130	159.78	8	0.8607	0.8607	3973876	0.8607	0.8607	0.7808	0.9052
6	99812	99812	131	0.321643369	2200	7	2459	4312	3390	130.19	10	0.8145	0.8080	42340268	0.8347	0.8080	0.6995	0.8487
7	43515	43515	122	0.487797882	1600	16	78053	1555	92	123.25	12	0.7288	0.7288	644453	0.7288	0.7288	0.4908	0.7350
8	1429	1429	128	0.533352314	1600	4	75464	3947	5802	128.50	12	0.8729	0.8729	3178245	0.8729	0.8729	0.7321	0.8729
9	32243	32243	131	0.585260305	1500	17	14629	8766	312	132.31	14	0.8347	0.8145	2805275	0.8211	0.8145	0.6436	0.8632
10	73119	73119	131	0.229810848	3500	16	9836	4948	1058	131.21	10	0.8142	0.7931	2051008	0.7863	0.7863	0.5869	0.8000
11	17999	17999	124	0.951335937	1800	4	78223	1788	1604	124.50	11	0.7647	0.7647	602173	0.7647	0.7521	0.5458	0.7778
12	28852	28852	126	0.158708840	1300	3	87647	149	3693	120.75	16	0.8120	0.7724	1063602	0.7724	0.7661	0.5624	0.7724
13	36616	36616	108	0.695777534	1500	7	81697	1836	5776	108.13	9	0.7250	0.7073	14945	0.7073	0.7016	0.4117	0.7016
14	36109	36109	155	0.113228910	5300	4	75932	454	825	154.54	11	0.8807	0.8205	203889	0.8000	0.8000	0.6244	0.8496
15	71743	71743	123	0.199108062	2600	16	78224	1225	2561	122.29	9	0.6917	0.6917	841593	0.6640	0.6640	0.4304	0.7034
16	93195	93195	132	0.063720763	1300	4	12421	4311	3193	128.75	11	0.8390	0.8390	4972129	0.8390	0.8390	0.6555	0.8390
17	44042	44042	148	0.092629000	4200	4	80684	820	1636	151.15	16	0.0000	0.0000	301162	0.0000	0.0000	0.6974	0.0000
18	96284	96284	135	0.440206336	2600	10	393	4151	3544	130.00	8	0.7920	0.7920	4339978	0.7920	0.7920	0.6173	0.8390
19	94740	94740	118	0.700516130	3100	4	13689	3833	3662	123.88	7	0.8468	0.8174	26649914	0.8174	0.7833	0.5909	0.7833
20	12700	12700	124	0.442402115	1800	4	81149	4326	1726	124.30	10	0.8319	0.8049	5110619	0.8049	0.7984	0.5756	0.8250

4.2 Feature Engineering

Beyond cleaning existing features, we created derived variables to capture non-linear relationships and interaction effects. Feature engineering can substantially improve model performance when domain knowledge suggests specific functional forms. While anonymized features limited theory-driven engineering, we applied general-purpose transformations.

Polynomial Features

Polynomial features capture non-linear relationships through power transformations. For instance, if loss has a quadratic relationship with feature f471 (e.g., moderate values minimize loss while extreme values increase it), a linear model using only f471 will underfit. Adding $f471^2$ allows fitting parabolic patterns.

We generated polynomial features by squaring the top 10 features identified through correlation analysis (f471, f536, f674, f67, f617, f599, f413, f68, f617, f620). This created 10 additional variables. Higher-order polynomials (cubic, quartic) were not pursued to avoid overfitting given the modest correlations observed.

Interaction Features

Interaction features capture synergistic effects where the combined influence of two variables differs from their additive contributions. Mathematically, an interaction between features x_1 and x_2 is represented as their product: $x_1 \times x_2$.

Financial examples abound: high debt might be manageable with high income but problematic with low income (debt × income interaction). Short loan terms might be fine for small balances but stressful for large balances (term × balance interaction). Without knowing feature meanings, we couldn't engineer specific domain-motivated interactions.

Instead, we created all pairwise interactions among the top 5 correlated features: (5 choose 2) = 10 interactions. This included: f471×f536, f471×f674, f471×f67, f471×f617, f536×f674, f536×f67, f536×f617, f674×f67, f674×f617, and f67×f617.

Combined with the 10 original top features and 10 polynomial terms, this yielded a 30-feature enhanced dataset for testing whether linear models benefit from explicit non-linearity and interaction encoding.

Automated Feature Selection via Lasso

While manual feature engineering creates specific transformations, automated feature selection identifies which of the hundreds of original features actually contribute to prediction. Lasso regression (Least Absolute Shrinkage and Selection Operator) accomplishes this through L1 regularization:

minimize: $||y - X\beta||^2 + \lambda||\beta||_1$

minimize: $||y - X\beta||^2 + \lambda||\beta||_1$

The L1 penalty $\lambda||\beta||_1$ (sum of absolute coefficient values) induces sparsity—coefficients of irrelevant features are shrunk to exactly zero. This contrasts with Ridge regression's L2 penalty, which shrinks coefficients toward zero but rarely sets them exactly to zero.

We applied Lasso with 10-fold cross-validation to select the optimal regularization parameter λ. The procedure:

1. Partitioned training data into 10 folds
2. For each λ value in a grid spanning 10^-3 to 10^2:
   a. Trained Lasso on 9 folds
   b. Evaluated MAE on the held-out fold
   c. Repeated for all fold combinations
3. Selected λ minimizing average cross-validation MAE

The optimal λ = 0.0288 yielded a model with 88 non-zero coefficients (88% dimensionality reduction from 731 features). This aggressive selection suggests most features contribute minimal information beyond noise. The 88 selected features formed a compact representation capturing the essential predictive signal.

4.3 Model Selection Strategy

We evaluated seven distinct modeling approaches selected to span different algorithmic paradigms:

Linear models (OLS, Lasso, Ridge, Elastic Net) assume linear relationships but differ in regularization. Tree-based models (Random Forest, XGBoost) can capture non-linearities and interactions without explicit feature engineering. Enhanced linear models test whether manually engineered non-linear features improve simple linear approaches.

This diversity ensures we explore various inductive biases: parametric vs. non-parametric, linear vs. non-linear, greedy vs. ensemble. If one model class substantially outperforms others, it suggests the data exhibit characteristics favoring that approach (e.g., tree superiority indicates non-linear patterns; Lasso success suggests sparse signal in high-dimensional space).

4.4 Evaluation Framework

Model evaluation followed standard machine learning protocol with training-validation splitting:

The 79,950 training observations were partitioned into training (70%, n=56,000) and validation (30%, n=24,000) subsets using stratified sampling to preserve the loss distribution. All models were trained exclusively on the training subset. Hyperparameter tuning (where applicable) used cross-validation within the training subset. The validation subset provided unbiased performance estimates, touched only once after all modeling decisions were finalized.

Three metrics assessed performance:

Mean Absolute Error (MAE) = $(1/n)\Sigma|y_i - \hat{y}_i|$ measures average prediction error in original units. MAE is interpretable, robust to outliers, and directly reflects business costs. Unlike squared errors (MSE, RMSE), MAE treats all errors equally rather than disproportionately penalizing large deviations.

Root Mean Squared Error (RMSE) = $\sqrt{[(1/n)\Sigma(y_i - \hat{y}_i)^2]}$ penalizes large errors more heavily through squaring. RMSE is less interpretable than MAE but reveals whether models struggle particularly with extreme values. RMSE ≥ MAE always holds, with larger gaps indicating heterogeneous error distributions.

R-squared = 1 - (SS_residual/SS_total) represents the proportion of variance explained. R^2 ranges from 0 (no better than predicting mean) to 1 (perfect prediction). In regression contexts, R^2 provides standardized assessment of model fit independent of scale.

5. Model Implementation

5.1 Linear Models

Ordinary Least Squares Baseline

We established a baseline using ordinary least squares (OLS) regression with the top 50 features ranked by correlation with loss. This subset size balances predictive power against

overfitting risk in a 56,000-observation training set (feature-to-observation ratio of 1:1,120, well within recommended guidelines).

The baseline achieved validation MAE of 1.434 and R^2 of 0.0045. While R^2 is low, the model outperforms the naive baseline (always predict mean loss) which yields MAE of 1.53. The 6% improvement over naive prediction demonstrates that features contain some signal, albeit weak. We found OLS an inadequate feature selection model due to multicollinearity between feature variance.

Lasso Regression

Lasso was implemented using glmnet's built-in cross-validation procedure, which performs 10-fold cross-validation by default to select the optimal regularization parameter. While the underlying process aligns with standard cross-validation methodology, our implementation relied on established library functions rather than manual iteration.

Lasso regression with 10-fold cross-validation selected $\lambda = 0.0288$ as optimal, producing a sparse model with 88 non-zero coefficients from 731 candidates. This aggressive selection reflects the weak individual feature-target correlations observed during exploration.

Validation performance: MAE = 1.428, RMSE = 4.149, $R^2 = 0.0060$. Lasso slightly outperformed OLS despite using 88 vs. 50 features, suggesting the manually-selected top-50 omitted some useful predictors that Lasso identified through its global optimization. The marginal R^2 improvement from 0.0045 to 0.0060 (33% relative gain) is meaningful given the low baseline.

Examining non-zero coefficients revealed several patterns. Feature f471 had the largest absolute coefficient (0.140), consistent with its strongest correlation. Features f674, f536, f67, and f617 all received substantial weights, confirming their importance. Interestingly, many features with near-zero individual correlations received non-zero coefficients, indicating they contribute predictive value in combination with other variables.

Ridge Regression

Ridge regression with L2 regularization selected $\lambda = 5.39$ through cross-validation. Unlike Lasso, Ridge retains all 731 features but shrinks coefficients proportionally based on predictive value. This approach may outperform Lasso when many features contribute small effects rather than a few features dominating.

Validation performance: MAE = 1.430, RMSE = 4.151, $R^2 = 0.0054$. Ridge performed nearly identically to Lasso, suggesting sparse and dense representations capture similar information. The fractional improvement over OLS indicates that using all features with appropriate regularization marginally beats manually selecting 50 features.

Elastic Net

Elastic Net combines L1 and L2 penalties with mixing parameter $\alpha = 0.5$, balancing Lasso's feature selection and Ridge's coefficient shrinkage. Cross-validation selected $\lambda = 0.0575$ as optimal.

Validation performance: MAE = 1.428, RMSE = 4.149, $R^2 = 0.0060$. Elastic Net matched Lasso's performance exactly, unsurprising given both incorporate L1 penalties driving sparsity. The hybrid approach offers no advantage here, likely because the problem doesn't exhibit strong multicollinearity where Elastic Net theoretically excels.

Enhanced Linear Model with Interactions

To test whether explicit non-linearity encoding improves linear models, we trained OLS on the 30-feature engineered dataset (top 10 features + 10 polynomial terms + 10 interactions).

Validation performance: MAE = 1.430, RMSE = 4.161, R^2 = 0.0036. The enhanced model performed slightly worse than standard OLS with top 50 features, indicating that polynomial and interaction terms did not capture meaningful patterns. This negative result suggests: (1) true non-linearities may be more complex than simple quadratic forms, (2) interactions among top correlated features don't drive loss variation, or (3) tree-based models capture such patterns more flexibly.

Linear modeling feature analysis

We initially focused on Lasso for feature selection but then branched out to try Elastic Net, Ridge, and PLS.

The MAEs for the linear models were very close, with Lasso's being the best followed closely by Elastic Net.

Model <chr>	MAE <dbl>
LASSO	1.429243
Ridge	1.432113
Elastic Net	1.429286
PLS	1.434224

We were curious to see if the same features were prioritized across dimensionality reduction models. Because Ridge and PLS perform regularization and weight shrinkage based on coefficient manipulation instead of feature elimination, we compared the top 20 highest weight features assigned by each for this comparison.

As expected, LASSO, Elastic Net, and Ridge shared many similar features. The top 20 prioritized features selected by Partial Least Squares (PLS), on the other hand, were not seen in any of the other models. This suggests the PLS components may offer unique insight based on feature transformation. PLS constructs components from combinations of variables, so its notion of feature importance is not directly comparable to models that select features based on individual performance. While none of the top 20 features contributing to the six selected PLS components overlapped with the other

models, this likely reflects a difference in how importance is distributed across combinations of variables rather than individual predictors.

Top features shared by LASSO, Elastic Net, and Ridge

f111, f218, f221, f313, f383, f384, f398, f413, f471, f526, f533, f536, f615, f663, f673, f674, f775, and f80

Features selected by LASSO alone

f113, f145, f199, f300, f720

Top PLS features prioritized by weight

Feature <chr>	Importance <dbl>
f545	0.3271145
f575	0.3255873
f540	0.3254141
f538	0.3249765
f539	0.3249521
f224	0.3249387
f225	0.3249387
f226	0.3249387
f227	0.3249387
f570	0.3239600
f568	0.3233780
f569	0.3233580
f254	0.3233439
f255	0.3233439
f256	0.3233439
f257	0.3233439
f584	0.3228900
f579	0.3213211
f577	0.3209775
f578	0.3209705

Top PLS components and the features transformed

```
$`1`
[1] "f9"   "f1"   "f727" "f777"

$`2`
 [1] "f78"  "f79"  "f95"  "f96"  "f97"  "f98"  "f105" "f106" "f107" "f108" "f437" "f439"
[13] "f445" "f446" "f447" "f467" "f478" "f493" "f494" "f503" "f504" "f505" "f511" "f606"
[25] "f607" "f608" "f7"   "f438" "f449" "f457" "f488" "f498" "f508" "f610" "f440" "f115"
[37] "f116" "f117" "f118" "f125" "f126" "f127" "f128" "f452" "f453" "f454" "f455" "f460"
[49] "f475" "f476" "f481" "f483" "f484" "f485" "f486" "f491" "f495" "f496" "f501" "f85"
[61] "f86"  "f87"  "f88"  "f506" "f735" "f717"

$`3`
 [1] "f214" "f215" "f216" "f234" "f235" "f236" "f237" "f244" "f245" "f246" "f247" "f264"
[13] "f265" "f266" "f267" "f519" "f521" "f532" "f543" "f548" "f549" "f550" "f553" "f555"
[25] "f558" "f559" "f560" "f563" "f565" "f573" "f577" "f578" "f579" "f582" "f584" "f8"
[37] "f520" "f254" "f255" "f256" "f257" "f538" "f539" "f540" "f545" "f568" "f569" "f570"
[49] "f575" "f224" "f225" "f226" "f227" "f523"
```

We considered it may be worthwhile to apply ensemble predictive analytics to both the LASSO and PLS manipulated data sets independently. They could both reveal important insight.

5.2 Tree-Based Models

Random Forest

Random Forest constructs an ensemble of decision trees, each trained on bootstrapped data samples and random feature subsets. Predictions are averaged across trees, reducing variance while maintaining low bias. We configured Random Forest with 100 trees, considering top 100 features per tree, sampling 10 features per split, and requiring minimum 50 observations per terminal node.

The feature limit (100 vs. 731 total) was imposed for computational tractability—training Random Forest on 731 features with 56,000 observations requires substantial memory and time. Preliminary experiments suggested marginal returns beyond 100 features given weak univariate relationships.

Validation performance: MAE = 1.519, RMSE = 4.178, R^2 = 0.0033. Random Forest underperformed linear models, contrary to expectations. Several factors may explain this: (1) feature limitation may have excluded important variables, (2) weak signals may not provide sufficient information for tree splitting, (3) hyperparameters may be suboptimal, or (4) the problem genuinely favors linear approaches. The lower R^2 despite higher MAE indicates Random Forest makes larger errors on some cases while being accurate on others (higher variance).

XGBoost (Gradient Boosting)

XGBoost implements gradient boosting with sophisticated regularization and optimization. Unlike Random Forest's parallel tree construction, XGBoost builds trees sequentially, each

correcting residual errors from the ensemble of previous trees. This iterative refinement often achieves superior accuracy.

We configured XGBoost with: maximum depth = 6 (limiting individual tree complexity), learning rate $\eta = 0.05$ (conservative step size balancing accuracy and convergence speed), subsample = 0.8 (training each tree on 80% of data to reduce overfitting), column subsample = 0.8 (considering 80% of features per tree), and minimum child weight = 5 (requiring at least 5 observations per leaf).

Early stopping monitored validation MAE during training, halting after 20 rounds without improvement. Training converged at iteration 140 with best validation MAE of 1.425.

Validation performance: MAE = 1.425, RMSE = 4.154, $R^2 = 0.0046$. XGBoost achieved the best MAE among all models, marginally outperforming Lasso (1.428) and substantially beating Random Forest (1.519). The improvement over linear models, while modest in absolute terms (0.003 MAE), represents 0.2% error reduction—meaningful when losses are monetary and portfolios are large.

Feature importance analysis from XGBoost confirmed f471 as the most important predictor (12% of total gain), followed by f674 (8%), f536 (6%), and f617 (5%). This alignment with correlation rankings validates both approaches. Notably, XGBoost identified several mid-importance features (f413, f620, f599) that correlation analysis ranked lower, illustrating how ensemble methods discover features that contribute through interactions rather than marginal effects.

5.3 Ensemble Approach

Ensemble methods combine predictions from multiple models to improve accuracy and robustness. When individual models make different types of errors, averaging can reduce overall error. We constructed an ensemble using inverse-MAE weighting, giving greater influence to better-performing models:

$$w_m = (1/MAE_m) / \Sigma(1/MAE_m)$$

$$w_m = (1/MAE_m) / \Sigma(1/MAE_m)$$

where w_m is the weight for model m and MAE_m is its validation error. We combined the three best-performing models: Lasso (MAE 1.428), XGBoost (MAE 1.425), and Random Forest (MAE 1.519), yielding weights:

- Lasso: w = 0.334
- XGBoost: w = 0.335
- Random Forest: w = 0.331

- Lasso: w = 0.334
- XGBoost: w = 0.335
- Random Forest: w = 0.331

The final prediction equals: $\hat{y} = 0.334(\text{Lasso}) + 0.335(\text{XGBoost}) + 0.331(\text{Random Forest})$

Validation performance: MAE = 1.455, RMSE = 4.148, R^2 = 0.0067. Paradoxically, the ensemble underperformed the best individual model (XGBoost), achieving worse MAE (1.455 vs. 1.425) despite higher R^2 (0.0067 vs. 0.0046). This pattern occurs when models are highly correlated—they make similar predictions and similar errors, so averaging provides no diversification benefit.

Examining prediction correlations confirmed this: Lasso and XGBoost predictions correlated at r = 0.92, XGBoost and Random Forest at r = 0.88, and Lasso and Random Forest at r = 0.86. With such high correlations, the ensemble is effectively a weighted average of nearly identical predictions. The Random Forest component, being less accurate and not sufficiently different, dilutes XGBoost's superior performance.

Nevertheless, we retained the ensemble for test predictions. Ensembles often generalize better even when validation performance appears worse, as averaging reduces sensitivity to validation set peculiarities. The improved R^2 (0.0067 vs. 0.0046) suggests the ensemble captures systematic patterns more consistently, even if individual predictions are marginally less accurate.

6. Results and Analysis

6.1 Model Performance Comparison

Table 1 summarizes validation performance across all eight models, ranked by MAE (primary metric):

Rank	Model	Validation MAE	Validation RMSE	Validation R^2
1	XGBoost	1.425	4.154	0.0046
2	Lasso	1.428	4.149	0.0060
3	Elastic Net	1.428	4.149	0.0060
4	Ridge	1.430	4.151	0.0054
5	Enhanced Linear (Interactions)	1.430	4.161	0.0036
6	Linear Baseline (Top 50)	1.434	4.153	0.0045
7	PLS	1.437	4.162	-0.00007
8	Random Forest	1.519	4.178	0.0033

—	Ensemble (Lasso+XGB+RF)	1.455	4.148	0.0067

Table 1: Model performance on held-out validation set (n=24,000)

Several patterns emerge from these results:

First, all models cluster within a narrow MAE range (1.425-1.519), suggesting we've reached the predictive ceiling given available features. The 6.6% gap between best (XGBoost 1.425) and worst (Random Forest 1.519) is modest compared to typical ML competitions where gaps exceed 20%.

Second, R-squared values are universally low (0.33%-0.67%), indicating that less than 1% of loss variance is explained by any model. This isn't failure—it reflects genuine data limitations. Financial losses depend on factors beyond loan characteristics: borrower life events (health crises, job loss), macroeconomic conditions (recessions, interest rate changes), and stochastic elements (luck, timing). Static features capture only a fraction of these dynamics.

Third, XGBoost's victory, though narrow, suggests weak non-linear patterns that gradient boosting exploits. If relationships were purely linear, Lasso would likely match or exceed XGBoost. The 0.003 MAE advantage implies non-linearities exist but are subtle.

Fourth, Random Forest's unexpected weakness (worst MAE, lowest R^2) likely stems from the 100-feature restriction. Random Forest excels when provided full feature sets to discover interactions. Limiting to 100 features may have excluded critical variables for tree splitting. This computational compromise sacrificed performance.

Fifth, manual feature engineering (polynomial/interaction terms) failed to improve linear models. The enhanced model's worse performance suggests either: (a) relevant non-linearities are too complex for quadratic forms, or (b) interactions among top correlated features aren't where predictive power resides. Tree methods like XGBoost capture non-linearities and interactions automatically without manual specification.

Sixth, RMSE values are all near 4.15 while MAE values are near 1.43. The ratio RMSE/MAE ≈ 2.9 exceeds the theoretical minimum of $\sqrt{2} \approx 1.41$ (achieved when all errors are identical), indicating heterogeneous errors. Some predictions are very accurate (small errors) while others are quite wrong (large errors). This heterogeneity is typical in skewed loss distributions where a few extreme cases dominate squared error metrics.

Seventh, PLS achieved similar MAE and RMSE to other linear models but exhibited near-zero (slightly negative) R^2, suggesting that compressing the data into a small number of components may discard weak but useful predictive signals.

Finally, while there is strong agreement across models on key features, further analysis suggests that many of these variables belong to highly correlated groups. As a result, different models may select different representatives of the same underlying signal.

6.3 Model Diagnostics

Beyond aggregate metrics, examining residuals (prediction errors) reveals model behavior patterns. We analyzed the XGBoost model (best performer) on validation data:

Residual distribution approximates normality with mean near zero (-0.002, effectively unbiased) but exhibits slight right skew. Most errors cluster around zero (median absolute residual 0.68), but the distribution has a long right tail with maximum error reaching 6.1. This asymmetry reflects the loss variable's right skewness—extreme losses are harder to predict than typical cases.

Plotting residuals versus fitted values reveals heteroscedasticity: error variance increases for higher predicted losses. For predicted losses below 1.0, residuals concentrate tightly around zero (±1). For predicted losses above 2.0, residuals spread more widely (±3 or more). This pattern is expected: high-loss cases involve more complex, variable circumstances, making precise prediction difficult.

Actual-versus-predicted scatter shows reasonable alignment along the diagonal for most observations. The correlation between actual and predicted values is 0.085 ($R = \sqrt{0.0046} \approx 0.07$ from R^2). While low in absolute terms, it indicates predictions aren't random—they track actual values weakly but systematically.

Error analysis by loss magnitude reveals interesting patterns:
- Low losses (0-1): Mean absolute error = 0.72, representing 72% average percentage error
- Medium losses (1-2): MAE = 1.15, representing 58% average percentage error
- High losses (2+): MAE = 2.84, representing 71% average percentage error

Error analysis by loss magnitude reveals interesting patterns:
- Low losses (0-1): Mean absolute error = 0.72, representing 72% average percentage error
- Medium losses (1-2): MAE = 1.15, representing 58% average percentage error
- High losses (2+): MAE = 2.84, representing 71% average percentage error

Medium-range losses are predicted most accurately (lowest percentage error), perhaps because they represent typical default scenarios with more consistent patterns. Very low losses are difficult because they're close to zero (small absolute errors become large percentage errors). High losses are difficult because they're rare and heterogeneous.

7. Discussion

7.1 Interpretation of Findings

Our results paint a nuanced picture of loan loss prediction. On one hand, we achieved modest success: MAE of 1.425 improves 7% over naive baseline (predicting mean) and identifies features with consistent importance across methods. On the other hand, R^2 below 1% indicates the vast majority of loss variance remains unexplained.

This paradox—meaningful prediction yet limited explanatory power—characterizes many financial applications. Unlike physics where laws explain 99%+ of variation, or marketing where demographics might explain 20-30%, financial outcomes involve fundamentally unpredictable elements. A borrower's job loss, health crisis, or divorce cannot be predicted from

origination-time features. Macroeconomic shocks (recession, interest spikes) affect all loans but occur irregularly. Even with perfect information about a borrower's current state, future events introduce irreducible uncertainty.

Yet our models provide value. In portfolio management, even weak signals aggregate profitably across thousands of loans. If a model identifies that loans with certain features average 10% higher losses, charging 10 basis points higher interest compensates for this risk. With $1 billion in loans, that's $1 million in additional revenue. The weak R^2 doesn't negate this business value.

The consistency across methods is encouraging. When correlated, Lasso, and XGBoost independently identify f471, f674, f536, f67, and f617 as top predictors, we can confidently report that these features—or closely related variables—capture important predictive signal. In practice (with unmasked features), this would inform underwriting guidelines: pay special attention to these characteristics when evaluating applications. However, many predictors appear highly correlated, suggesting these features may represent broader groups rather than uniquely important variables.

The failure of polynomial and interaction features deserves discussion. We hypothesized that manually engineering non-linearities might improve linear models, yet the enhanced model underperformed. Several interpretations are plausible:

First, true non-linearities may be more complex than quadratic forms. If the relationship between f471 and loss involves thresholds, discontinuities, or higher-order effects, simple polynomials won't capture it. XGBoost succeeds because decision trees naturally handle arbitrary functional forms.

Second, our interaction terms (f471×f536, etc.) may not reflect actual interactions. Interactions are highly specific; the right combinations matter. With 731 features, there are 267,000+ pairwise interactions. We tested only 10 based on marginal correlations. The interactions that truly matter might involve features with weak marginal effects.

Third, in sparse-signal settings (our case), adding features risks overfitting. Polynomial and interaction terms are additional parameters to estimate. With weak signals, the model may fit noise in these engineered features rather than true patterns. The slight performance decline suggests this occurred.

Finally, given the similar MAEs across models and the presence of many features with overlapping predictive value, a feature transformation approach like PLS may provide additional insight into underlying relationships among correlated variables. Although its predictive performance is slightly lower, PLS helps reveal how groups of features collectively contribute to loss. However, it would not be sufficient alone as it appears to have discarded some weaker but useful signal.

7.2 Limitations

Several limitations constrain our analysis and should inform interpretation:

Data anonymization prevents domain knowledge application. Without knowing which features represent income, debt, credit score, loan amount, etc., we cannot engineer theoretically-motivated transformations. Financial intuition suggests debt-to-income ratios, payment-to-income ratios, and loan-to-value metrics would be highly predictive—but we cannot

compute these from anonymous features. This limitation likely degrades performance substantially compared to what domain experts could achieve with identified variables.

Missing variables constitute another major constraint. Important predictors absent from the dataset might include: employment industry (vulnerable sectors like oil/gas or retail), geographic location (regional economic conditions), temporal effects (origination during recession vs. expansion), macroeconomic indicators (interest rates, unemployment), and behavioral data (payment history, credit utilization). Static borrower characteristics at origination cannot capture dynamics that unfold post-origination.

Computational constraints affected Random Forest evaluation. Restricting to 100 features rather than all 731 was necessary for timely execution but potentially harmed performance. This reflects practical ML trade-offs: exhaustive search over all hyperparameters and feature combinations would require weeks of computation. We prioritized breadth (seven model types) over depth (extensive tuning of each).

The evaluation metric (MAE) treats all errors equally. In practice, financial institutions may have asymmetric loss functions: underestimating a $10 loss costs little, but underestimating a $1,000 loss costs greatly. A custom loss function weighting large losses more heavily might produce different optimal models. We used MAE for simplicity and interpretability, but acknowledge this assumption.

Temporal validation is absent. Our random train-validation split assumes loans are exchangeable across time. If loss patterns shift over time (economic cycles, regulatory changes, lending standard evolution), models trained on historical data may not generalize to future loans. Ideally, we'd validate on genuinely future data (train on 2010-2015, validate on 2016-2017). The dataset lacks temporal information preventing this analysis.

Finally, we lack business context for interpreting feature importance. Knowing f471 is most predictive doesn't inform decisions without knowing what f471 represents and whether it's actionable. If f471 is credit score, we can use it in underwriting. If f471 is age, legal constraints may prohibit its use. Feature interpretation requires domain knowledge we don't possess.

7.3 Business Implications

Despite analytical limitations, our models have practical applications:

Portfolio risk assessment: Aggregating predicted losses across a loan portfolio estimates expected losses, informing reserve requirements. Regulatory frameworks (Basel III) require banks to hold capital against credit risk. Our model's predictions could calibrate these reserves. Even with R^2 below 1%, aggregation across thousands of loans reduces error through diversification.

Risk-based pricing: Loans with higher predicted losses should command higher interest rates to compensate lenders. If our model predicts loan A will lose 0.5 on average while loan B will lose 1.0, charging an extra 50 basis points on loan B maintains profitability. This differential pricing aligns costs with risks, improving portfolio returns.

Underwriting guidelines: Features identified as important (f471, f674, f536) could receive extra scrutiny during loan approval. Alternatively, applicants with concerning values on these features might require compensating strengths (higher down payment, co-signer) to offset elevated risk. Feature importance focuses underwriter attention where it matters most.

Portfolio composition: If certain feature combinations predict systematically higher losses, avoiding concentration in those segments reduces portfolio risk. For example, if loans with high f471 and low f536 perform poorly, limiting exposure to this combination improves stability. This is the credit equivalent of equity portfolio diversification.

Early warning systems: Monitoring whether actual losses exceed predictions can signal deteriorating conditions requiring intervention. If predicted losses average 1.0 but actual losses spike to 2.0, economic conditions may have shifted, warranting tighter lending standards or increased reserves.

Competitive advantage: Even marginal improvements in loss prediction compound over time. Consider two banks with $10 billion loan portfolios: Bank A uses our model (MAE 1.43), Bank B uses naive prediction (MAE 1.53). Bank A's superior prediction enables better pricing, potentially improving profitability by $10 million annually (1% of portfolio × 0.10 MAE advantage). Over a decade, this compounds to substantial competitive advantage.

8. Conclusion and Future Work

This research developed and evaluated machine learning models for predicting loan default losses using a dataset of 80,000 historical records. Our comprehensive methodology encompassed data preprocessing (handling 587,747 missing values, removing zero-variance features, standardizing with outlier capping), feature engineering (polynomial terms, interactions, Lasso selection), and multi-model evaluation (seven approaches spanning linear, tree-based, and ensemble methods).

XGBoost achieved the best performance with validation MAE of 1.425, marginally outperforming Lasso (1.428) and substantially beating Random Forest (1.519). However, all models exhibited low R-squared values (0.3-0.7%), reflecting inherent unpredictability in financial losses beyond what static borrower features capture. This finding aligns with financial economics literature documenting that loan outcomes depend on post-origination events (macroeconomic shocks, borrower life events) not predictable from origination-time data.

Feature importance analysis revealed strong consensus across methods. Features f471, f536, f674, f67, and f617 consistently ranked among top predictors whether assessed via correlation, Lasso selection, or XGBoost importance. This convergence provides confidence these features genuinely drive loss variation. Lasso's aggressive feature selection (88 of 731 features) demonstrates that most variables contribute negligible information beyond noise.

Manual feature engineering (polynomial and interaction terms) failed to improve linear models, suggesting important non-linearities either exceed simple quadratic forms or XGBoost captures them more flexibly. The ensemble approach likewise underperformed the best individual model (XGBoost) due to high correlation among constituent model predictions, providing limited diversification benefit.

Despite modest explanatory power, our models provide practical value for financial institutions. Applications include portfolio risk assessment, risk-based pricing, underwriting guideline development, and early warning systems. In high-volume lending, even weak signals aggregate profitably across thousands of loans.

Future research directions include:

Deep learning architectures: Neural networks with multiple hidden layers might capture complex non-linearities beyond gradient boosting's capabilities. However, careful regularization (dropout, early stopping) would be essential given limited training data relative to parameter count.

Temporal modeling: If loan origination dates and economic indicators were available, incorporating time effects (recession indicators, interest rate environments, seasonal patterns) could substantially improve predictions. Time-series cross-validation would better assess future performance.

Two-stage modeling: Building separate models for default probability and loss-given-default, then combining their predictions, might outperform single-stage approaches. Different features may predict whether a loan defaults versus how much is lost conditional on default.

Alternative loss functions: Custom objective functions weighting large errors more heavily would better align with business costs. Banks care more about avoiding catastrophic losses than precisely predicting small losses. Asymmetric penalties could reshape model priorities.

Uncertainty quantification: Providing prediction intervals (80% confidence that loss will be X±Y) would be more useful than point predictions for risk management decisions. Quantile regression or conformal prediction could furnish these intervals.

Feature interpretation via domain experts: Collaborating with lending professionals to interpret anonymous features might enable theory-driven feature engineering. Understanding which features represent which concepts would unlock domain knowledge currently inaccessible.

In conclusion, while loan loss prediction faces fundamental limits due to inherent uncertainty in financial outcomes, machine learning methods can extract meaningful signals from high-dimensional borrower and loan characteristics. Our analysis demonstrates that systematic methodology—careful preprocessing, diverse model evaluation, rigorous validation—produces models with practical business value despite limited explanatory power. The key insight is not that we can predict losses precisely, but that we can identify systematic patterns that inform better decisions than intuition or simple heuristics alone.